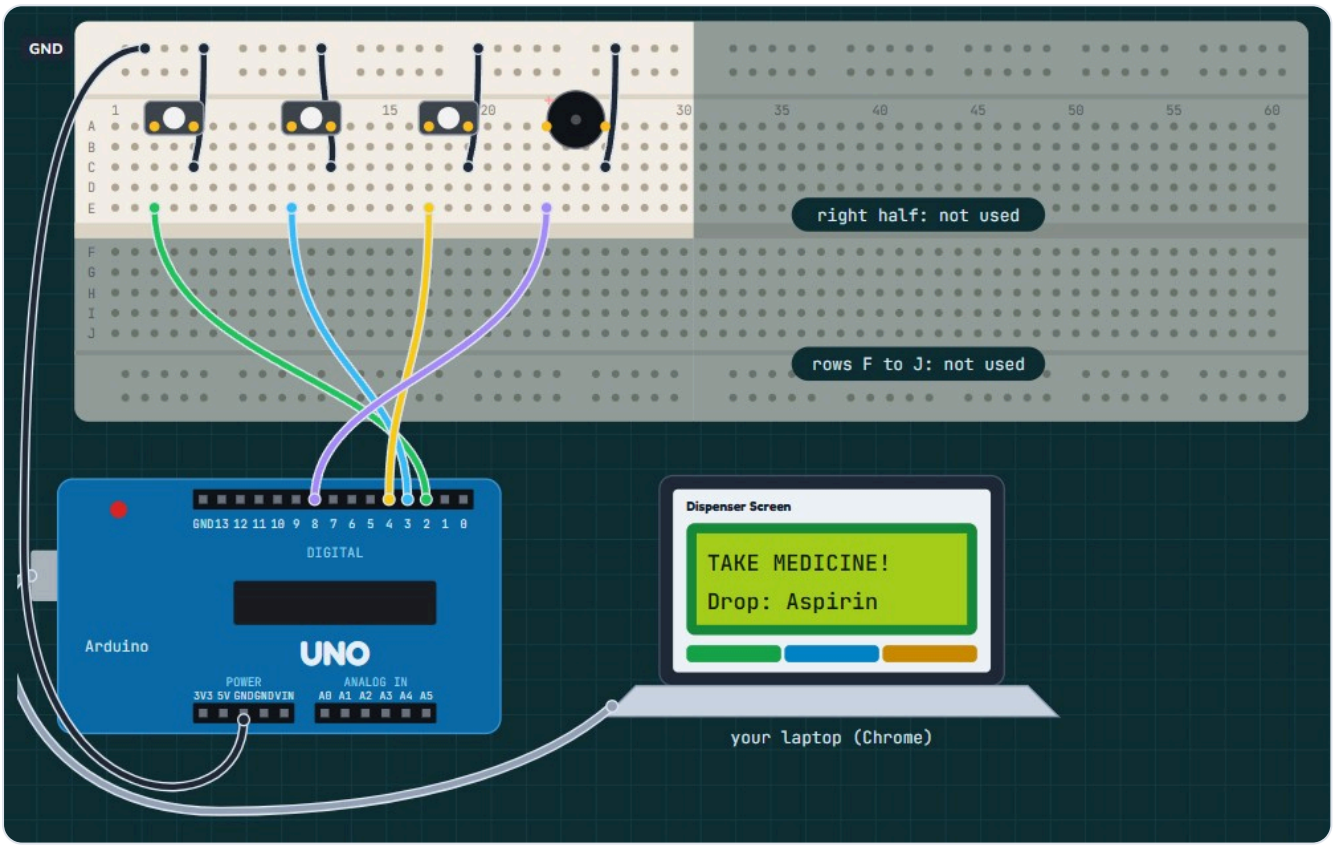
ARDUINO · HEALTHCARE · PROJECT GUIDE

Smart Medicine Dispenser

Every part of the project explained simply: what it is, how it works, and how to present it.



Inside

1 The project in one minute
2 The problem and our idea
3 The big picture
4 Every part explained
5 The wiring explained
6 What happens, step by step
7 The menu
8 How the Arduino code works
9 How the laptop screen works
10 The physics behind it
11 How to run the demo
12 Common questions and answers

PART 1

The project in one minute

If someone asks "what is this project?", this is the short answer.

> **Say this**
> "Our project is a Smart Medicine Dispenser for Alzheimer's patients and older people who forget their medicine. It's built on an Arduino Uno with three buttons and a buzzer, and it uses a laptop as its screen. It keeps the time for each medicine. When it's time, the buzzer rings, the laptop screen turns red and shows the medicine name, and the laptop also says the name out loud. Then it reduces the pill count and saves the dose in a log, so the caretaker can see which medicine was taken and when."

The 5 things it does

1. **Remembers** the time of each medicine (Aspirin 08:00, Panadol 14:00, Vitamin C 20:00).
2. **Alerts** the patient at that time with a sound, a red screen and a voice.
3. **Tells which medicine** to take, by name.
4. **Counts** the pills left in each compartment and warns when one is empty.
5. **Keeps a log** of every dose with the time, for the caretaker.

What we used

Part	Quantity	Job in one line
Arduino Uno	1	The brain. Runs the program.
Push buttons	3	SET, UP and DOWN, to use the menu.
Buzzer	1	Makes the alarm sound.
Breadboard + jumper wires	1 + some	Connect everything without soldering.
USB cable	1	Power and data between the Arduino and the laptop.
Laptop with Google Chrome	1	The screen, the clock, the voice and the log.

PART 2

The problem and our idea

The problem

Alzheimer's disease slowly takes away a person's memory. One of the first daily things they start forgetting is their medicine. This causes two kinds of mistakes:

- **Missed dose:** they forget to take the medicine at all.
- **Double dose:** they forget they already took it and take it again.

Many older people who take several medicines at different times have the same problem. Usually a son, daughter or caretaker keeps reminding them, but they can't be with the patient all day. And the patient can't rely on their own memory, because that's exactly the problem.

Our idea

Make a device that **remembers for the patient**. The patient doesn't need to remember anything; the device tells them when it's time and what to take. And the caretaker can look at the log later to see what happened.

Why these features suit Alzheimer's patients

Feature	Why it helps
Loud buzzer alarm	Gets their attention even if they are not looking at the screen.
Big red screen with the medicine name	Easy to read. They don't have to remember which pill.
Voice says the name	Helps if their eyesight is weak or they get confused by text.
Pill count	The caretaker knows when to refill the box.
Log with times	The caretaker can check if a dose was taken or missed.

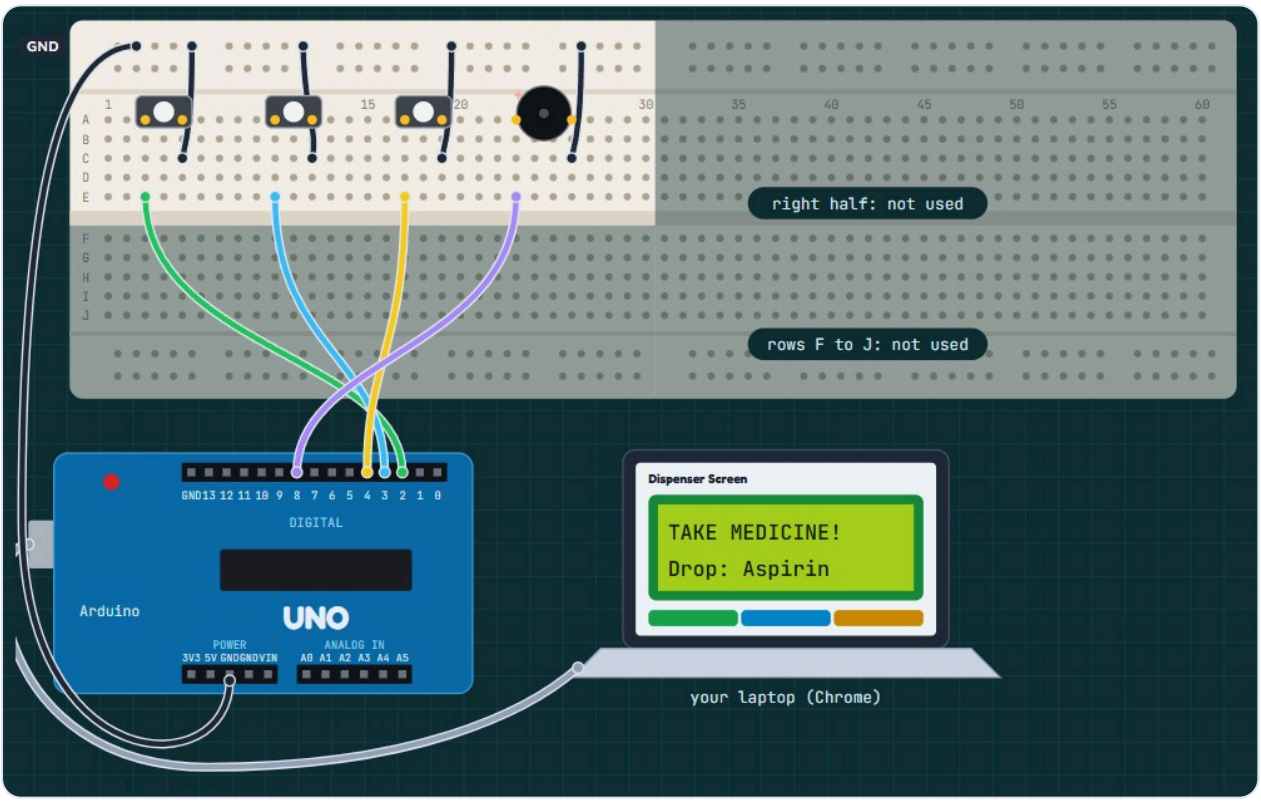
PART 3

The big picture

The whole project is three blocks talking to each other.



- **Input:** the buttons. The user presses them to use the menu.
- **Processing:** the Arduino. It runs our program, keeps the clock and checks the schedule.
- **Output:** the buzzer (sound) and the laptop screen (text, colours and voice).
- **Two-way link:** the Arduino sends what the screen should show; the laptop sends the real time back.



The final circuit. Each coloured wire is one signal. Black wires are ground (GND).

PART 4

Every part explained

Arduino Uno

What A small computer on a board (ATmega328P chip, 16 MHz).

Job Runs our program. Keeps the time, reads the buttons, rings the buzzer, sends screen text to the laptop.

Why Easy to program, works on 5 V, and has many pins for buttons and the buzzer.

Push buttons ×3

What Small switches. Pressing joins their two legs.

Job SET (pin 2) opens/confirms. UP (pin 3) and DOWN (pin 4) move and change numbers.

Why Simple for an older person or caretaker to use.

Buzzer

What A piezo buzzer: a disc that vibrates when voltage changes.

Job Two-tone alarm at dose time, a click on each button press, a low warning if empty (pin 8).

Why Sound gets attention even when the patient isn't looking.

Laptop + screen page

What A page (Dispenser_Screen.html) opened in Chrome.

Job Shows the big LCD-style screen, next dose, pills left and the log. Speaks the medicine name. Sends the real time.

Why Big clear screen, voice, and an accurate clock, using a laptop we already have.

USB cable

What The normal Arduino cable (flat end to laptop, square end to Arduino).

Job Gives the Arduino 5 V power and carries data both ways.

Why One cable does both jobs, so no battery is needed.

Breadboard + wires

What A board with holes joined inside in groups of 5.

Job Connects the buttons and buzzer to the Arduino.

Why No soldering. Parts can be moved and fixed easily.

Pin summary

Arduino pin	Connected to	Type
2	SET button	Input
3	UP button	Input
4	DOWN button	Input
8	Buzzer (+ leg)	Output
GND	GND line on the breadboard (all minus sides)	Ground (0 V)
USB	Laptop	Power + data

PART 5

The wiring explained

How the breadboard works

- Every hole has an address: a **letter** (row A to J) and a **number** (1 to 60). For example `A3`.
- The 5 holes under one number in rows A to E (A3, B3, C3, D3, E3) are **joined inside** by a metal strip. Anything in them is connected.
- The long line of holes along the top edge is one long strip. We connected it to the Arduino's GND, so it is our **GND line**.
- We only used the left half (numbers 1 to 30) and rows A to E.

Every connection

Part	Legs in	Signal wire	Ground wire
GND line	-	-	Arduino GND → top edge line
SET button	`A3, A5`	`E3 → pin 2`	`C5 → GND line`
UP button	`A10, A12`	`E10 → pin 3`	`C12 → GND line`
DOWN button	`A17, A19`	`E17 → pin 4`	`C19 → GND line`
Buzzer	`A23 (+), A26`	`E23 → pin 8`	`C26 → GND line`

Why each button is wired this way

One leg of the button connects to an Arduino pin, the other leg to GND. When the button is **not pressed**, the pin is pulled up to 5 V by a resistor inside the Arduino, so it reads **HIGH**. When it is **pressed**, the two legs join and the pin is connected straight to GND, so it reads **LOW** (0 V). The program watches for this change from HIGH to LOW.

Why the buzzer is wired this way

The plus leg goes to pin 8 and the minus leg to GND. When the program wants a sound, pin 8 switches between 5 V and 0 V very fast (hundreds of times per second). That makes the buzzer vibrate and produce a sound.

> **Why no resistors?** The buttons use the Arduino's built-in pull-up resistors (turned on in the code with `INPUT_PULLUP`), and the buzzer draws very little current, so no extra resistors were needed.

PART 6

What happens, step by step

From plugging it in to the patient getting the medicine.

1. **Plug in.** The USB cable powers the Arduino. It starts the program, beeps twice and shows "Med Dispenser, Starting...".
2. **Connect the screen.** On the laptop, the Dispenser Screen page is opened in Chrome and "Connect Arduino" is clicked. The two can now talk.
3. **Clock is set.** The laptop sends its real time to the Arduino (for example 19:40). The Arduino's clock is now correct.
4. **Schedule is shown.** The Arduino sends the list of medicines, times and pill counts. The laptop shows them in the Medicine box and the Next dose panel.
5. **Menu.** The screen shows MENU. The caretaker can check or change times (menu 2) and pill counts (menu 3).
6. **Waiting.** In "1 Dispense" mode, the Arduino checks the time every minute and compares it with each medicine's time.
7. **Dose time.** When the time matches, the buzzer plays the alarm, the laptop screen turns red with "TAKE MEDICINE" and the name, and the laptop says "It is time to take your Aspirin".
8. **After the dose.** The pill count goes down by one (5 → 4), the screen turns green ("DONE, 4 left") and the dose is written in the log with the time.
9. **Empty box.** If a medicine's count is 0 at its time, the buzzer gives a low warning sound and the laptop says the medicine is empty and needs a refill.

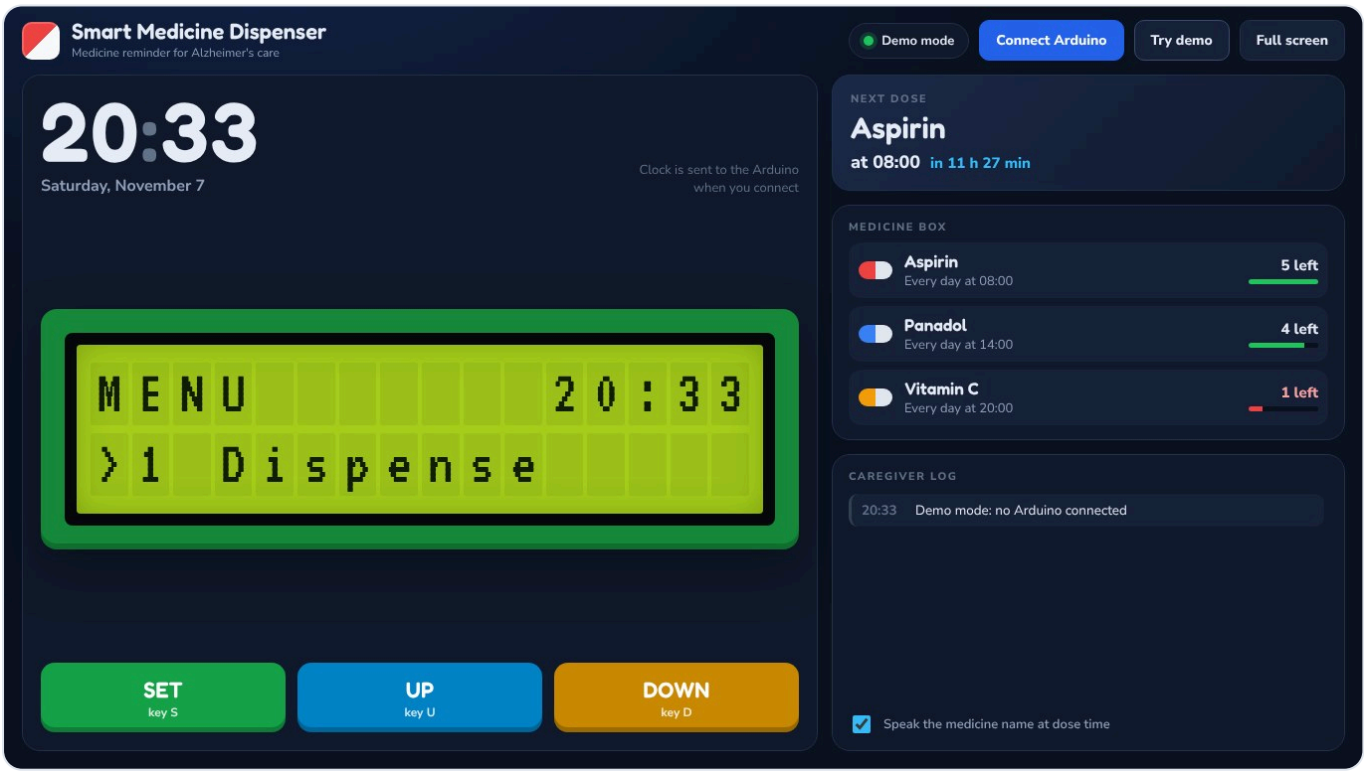
PART 7

The menu

The screen works like a 16×2 LCD: 2 lines, 16 characters each. UP and DOWN move between options, SET opens one.

Option	Screen shows	What the buttons do
Main menu	`MENU 19:40` `>1 Dispense`	UP / DOWN choose an option. SET opens it.
1 Dispense (normal mode)	`Next:Aspirin` `Dose 08:00 19:40`	Waits for dose times and rings the alarm. SET goes back to the menu.
2 Time Slots	`Set Aspirin` `Dose hour: 08`	UP / DOWN change the hour. SET saves it and moves to the next medicine. After the last one: "Saved!".
3 Medicines	`Aspirin Stk:05` `UP+ DN- SET>`	UP adds a pill, DOWN removes one. SET moves to the next medicine. After the last one: "Saved!".
Alarm	`TAKE MEDICINE!` `Drop: Aspirin`	Shown by itself at dose time. No button needed.

> The alarm only rings in **1 Dispense** mode. That's on purpose: while the caretaker is changing settings in menu 2 or 3, the device doesn't suddenly start ringing.



The laptop screen: clock and virtual LCD on the left, next dose, medicine box and caretaker log on the right.

PART 8

How the Arduino code works

Every Arduino program has two main parts: `setup()` runs **once** when the board starts, and `loop()` runs **again and again**, thousands of times per second.

setup(): getting ready

```
pinMode(BTN_SET, INPUT_PULLUP);   // buttons are inputs with the built-in pull-up resistor
pinMode(BUZZER, OUTPUT);          // buzzer pin is an output
setClock(7, 59, 0);               // start at 07:59 until the laptop sends the real time
show("Med Dispenser", "Starting...");
tone(BUZZER, 1000, 150);          // two start-up beeps
```

loop(): four jobs, over and over

```
void loop() {
  readLaptop();      // 1. read any message from the laptop (time, button presses)
  handleButtons();   // 2. check if SET, UP or DOWN was pressed
  checkSchedule();   // 3. is it time for a medicine?
  drawScreen();      // 4. send what the screen should show to the laptop
}
```

The medicine data

```
const char* medName[3] = {"Aspirin", "Panadol", "Vitamin C"};
byte doseHour[3]       = {8, 14, 20};   // dose times
byte stock[3]          = {5, 5, 5};     // pills in each compartment
```

How the clock works

The Arduino has no real clock chip. It uses `millis()`, which counts milliseconds since it started. When the laptop sends the time, the Arduino saves it, and from then on **time = saved time + seconds passed**. `CLOCK_SPEED` multiplies the seconds: 1 is real time, 60 makes 1 second count as 1 minute for the demo.

How a button press is detected

The code compares the pin's reading with its last reading. If it changed from HIGH to LOW, it counts as one press. It ignores changes within **40 ms** of the last one, because a button's metal contacts bounce for a few milliseconds when pressed (this is called **debouncing**).

How dose time is checked

```
if (lastCheckedMinute == doseHour[i] * 60) dispense(i);   // e.g. 8 * 60 = minute 480 = 08:00
```

Time is turned into "minutes since midnight" (08:00 = 480). Every new minute is checked against each medicine. If it matches and the mode is Dispense, `dispense()` runs.

What dispense() does

1. If the stock is 0: send `EMPTY`, low warning beeps, stop.
2. Send `ALARM` to the laptop and show "TAKE MEDICINE!".
3. Play the alarm: 880 Hz and 1320 Hz, three times.
4. Reduce the stock by 1 and send `TAKEN` with the new count.

PART 9

How the laptop screen works

The Arduino and the laptop talk through the USB cable using **serial communication** at **9600 bits per second**. Every message is one line of text. The page uses a Chrome feature called **Web Serial** to open the USB port, read the lines and send lines back.

Messages from the Arduino to the laptop

Message	Example	What the page does
`LCD`	`LCD|MENU 19:40▷1 Dispense`	Draws the two lines on the green screen.
`MED`	`MED|0|Aspirin|8|5`	Updates a medicine in the Medicine box (name, hour, stock).
`ALARM`	`ALARM|Aspirin`	Red full screen, chime, speaks the name, adds a log line.
`TAKEN`	`TAKEN|Aspirin|4`	Green "DONE" screen with pills left, adds a log line.
`EMPTY`	`EMPTY|Panadol`	Log warning and says "Panadol is empty. Please refill."
`LOG`	`LOG|Time slots saved`	Adds the text to the caretaker log.

Messages from the laptop to the Arduino

Message	Meaning
`T,19,40,05`	Set the clock to 19:40:05 (sent on connect and every 10 minutes).
`S / U / D`	SET / UP / DOWN pressed on the page or keyboard. Works like the real buttons.
`?`	"Send me everything again" (screen and medicine list).

The parts of the page

- **Clock and date:** taken from the laptop.
- **Virtual LCD:** shows exactly what the Arduino sends.
- **SET / UP / DOWN buttons:** copies of the real buttons (keys S, U, D also work).
- **Next dose:** which medicine is next and how long until it.
- **Medicine box:** each medicine with its time and a stock bar (red when 1 or fewer left).
- **Caretaker log:** every event with the time.
- **Voice:** uses the laptop's built-in speech to say the medicine name.
- **Try demo:** plays a sample alarm without the Arduino (backup for the presentation).

> Only one program can use the USB port at a time. Close the Arduino Serial Monitor before connecting the page, and click **Disconnect** on the page before uploading new code.

PART 10

The physics behind it

The physics concepts used in the circuit.

1. Digital voltage levels

The Arduino works with two voltage levels: **5 V = HIGH (1)** and **0 V = LOW (0)**. Buttons are read as HIGH or LOW; the buzzer pin is switched between HIGH and LOW. The 5 V comes from the laptop's USB port, so the whole circuit runs from one supply (power P = V × I) and no battery is needed.

2. Pull-up resistor and Ohm's law (the buttons)

Inside the Arduino, each button pin is connected to 5 V through a resistor of about 20 kΩ to 50 kΩ.

- **Not pressed:** no current flows, so there is no voltage drop across the resistor and the pin is at 5 V (HIGH).
- **Pressed:** the pin is connected to 0 V, and current flows through the resistor:

$$I = V / R = 5\ V / 20\ k\Omega = 0.25\ mA \quad \text{(and 0.1 mA for 50 k}\Omega\text{)}$$
$$P = V^2 / R = 25 / 20\,000 = 1.25\ mW$$

This is far below the 20 mA safe limit of a pin, so the circuit is safe without extra resistors. Without the pull-up the pin would "float" and read random values.

3. Piezoelectric effect and sound (the buzzer)

A piezo buzzer has a ceramic disc. When a voltage is applied, the disc bends slightly (the **converse piezoelectric effect**). Switching the voltage on and off quickly makes the disc vibrate, and the vibration pushes the air, making a **sound wave** at the same frequency.

$$T = 1 / f \text{ (period)} \qquad \lambda = v / f \text{ (wavelength, v = 343 m/s in air)}$$

Sound	Frequency f	Period T	Wavelength λ
DOWN button	600 Hz	1.67 ms	0.57 m
Alarm tone 1	880 Hz	1.14 ms	0.39 m
SET button	1000 Hz	1.00 ms	0.34 m
Alarm tone 2	1320 Hz	0.76 ms	0.26 m
UP button	1500 Hz	0.67 ms	0.23 m

Higher frequency = higher pitch. The alarm switches between 880 and 1320 Hz (ratio 1.5) so it sounds like a clear "beep-boop" that is easy to notice.

4. Serial communication (Arduino ↔ laptop)

Data is sent as a series of voltage pulses, one bit at a time, at **9600 bits per second**. Each character takes 10 bits (1 start + 8 data + 1 stop):

9600 ÷ 10 = 960 characters per second → one screen message (about 38 characters) takes about **40 ms**

PART 11

How to run the demo

For a demo we use the demo-speed code, where 1 second = 1 minute, so the alarm can be shown in about a minute.

Before the demo

1. Upload the demo-speed code (`CLOCK_SPEED = 60`). Wait for "Done uploading".
2. Close the Serial Monitor.
3. Open **Dispenser_Screen.html** in Chrome, click **Full screen**, then **Connect Arduino** and pick the Arduino's COM port.
4. Check: the screen shows MENU with the real time, and the medicine box is filled.

During the presentation (about 3 minutes)

1. **Explain the problem** in two lines (Part 1).
2. **Show the menu:** press UP and DOWN on the real buttons. Point out that the laptop screen changes straight away.
3. **Show menu 3:** change a pill count and show the medicine box updating.
4. **Set an alarm:** menu 2, set Aspirin's hour to one hour after the current clock, press SET three times to save.
5. **Choose 1 Dispense** and wait. In under a minute the buzzer rings, the screen turns red and the laptop says the medicine name.
6. **Show the result:** green "DONE, 4 left", and the new line in the caretaker log.
7. **Finish** with how it helps the patient and the caretaker, and the future work (app notifications, watch, all-in-one box).

> **If something fails during the demo:** click **Try demo** on the screen page. It runs the full alarm with a sample medicine, without the Arduino. If an upload fails with "not in sync", close and reopen the Arduino IDE.

PART 12

Common questions and answers

Q. What problem does your project solve?

A. Alzheimer's patients and older people forget to take their medicine or take it twice. Our device reminds them at the right time, tells them which medicine, and keeps a log for the caretaker.

Q. What is the role of the Arduino?

A. It's the controller. It keeps the time, stores the schedule and pill counts, reads the buttons, rings the buzzer and sends the screen text to the laptop.

Q. Why did you use the laptop as the screen?

A. It gives a much bigger and clearer screen, it can say the medicine name out loud, it keeps a log, and it sends the real time to the Arduino. The small LCD we tried only showed blocks and no text, so we moved the display to the laptop.

Q. How does the Arduino know the time?

A. It counts milliseconds with millis() from the moment it starts. When the laptop connects, it sends its real time, and the Arduino adds the time passed to that.

Q. How does a push button work in your circuit?

A. One leg goes to a pin, the other to GND. The internal pull-up resistor keeps the pin at 5 V (HIGH). Pressing connects the pin to 0 V (LOW). The code detects the change from HIGH to LOW.

Q. What is a pull-up resistor and why is it needed?

A. A resistor between the pin and 5 V. Without it the pin would "float" and read random values when the button isn't pressed. We used the Arduino's built-in one (INPUT_PULLUP), about 20–50 kΩ.

Q. Calculate the current when a button is pressed.

A. $I = V / R = 5\ V / 20\ k\Omega = 0.25\ mA$. That's very small and safe (the pin limit is 20 mA).

Q. How does the buzzer produce sound?

A. By the piezoelectric effect. The changing voltage from pin 8 makes a ceramic disc vibrate, which pushes the air and creates a sound wave at the same frequency.

Q. What frequencies does the alarm use? Find the wavelength.

A. 880 Hz and 1320 Hz. $\lambda = v / f = 343 / 880 = 0.39$ m and $343 / 1320 = 0.26$ m.

Q. What is debouncing?

A. When a button is pressed, its contacts bounce and switch on and off for a few milliseconds. The code ignores changes within 40 ms so one press is counted only once.

Q. How do the Arduino and laptop communicate?

A. Through serial communication over USB at 9600 bits per second. The Arduino sends lines like "ALARM|Aspirin", and the laptop sends lines like "T,19,40,05" for the time.

Q. How fast is that data transfer?

A. Each character is 10 bits, so 9600 / 10 = 960 characters per second. One screen message (about 38 characters) takes about 40 ms.

Q. Why does the alarm only ring in Dispense mode?

A. So it doesn't interrupt the caretaker while they are changing the times or pill counts in the other menus.

Q. What happens if a medicine runs out?

A. At its time, the buzzer gives a low warning, the screen shows it is empty, and the laptop says it needs a refill. It also shows "EMPTY" in the medicine box.

Q. Why is the demo clock faster?

A. Real doses are hours apart. With CLOCK_SPEED = 60, one second counts as one minute, so we can show the full alarm in about a minute. For real use it's set back to 1.

Q. Why didn't you use resistors with the buttons?

A. The Arduino has pull-up resistors built in, which we turned on with INPUT_PULLUP in the code.

Q. What problems did you face?

A. An upload error ("not in sync"), which was fixed by restarting the Arduino IDE. The LCD only showed blocks, so we used the laptop as the screen. Testing each part separately helped us find each problem quickly.

Q. What would you add in the future?

A. A mobile app that notifies the caretaker at dose time, a watch or band that reminds the patient, and the whole setup in one box with its own screen instead of a laptop. For Alzheimer's patients the device itself can stay very simple, just a buzzer, while the details go to the caretaker's phone.